

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконав:

студент групи АІ-245

Слюсаренко Д.Р.

Перевірили:

ас. Баранов О.М.

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Було проведено тестування продуктивності моделей qwen3:4b та llama3.2:3b за допомогою HTTP-запитів до API Ollama (/api/generate). Дослідження складалося з двох етапів: перевірки реакції на прямий фактологічний запит та серії з 5 запусків задачі з багатокроковою логікою. Щоб виключити вплив ініціалізації на метрики, перед зняттям замірів було виконано попередній розігрів моделей..

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ
eval_count	2246	5
Час, с	35,87	7,00
Токенів/с	73,31	68,44

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	2 / 5
eval_count	1283,2	185,2
Час, с	18,21	2,87

Типова помилка інструктивної моделі: модель не враховує логіку взаємозв'язку різниці та суми, видаючи хаотичні значення 50 грн, 55 грн, 100 грн, до того ж регулярно змішує мови «costs ручки need зазначати», «together обошлись».

Відповіді на питання (крок 6)

1. Я порівнював міркувальну модель (qwen3:4b) та інструктивну (llama3.2:3b) на простому фактичному питанні. Відповідь обидві дали однаково, але міркувальна згенерувала в десятки разів більше токенів. Куди пішли ці токени?

Відповідь: Усі 2246 токенів моделі qwen3:4b пішли на генерацію внутрішнього прихованого монологу (Chain-of-Thought) у блоці thinking. Модель

не просто витягла факт із пам'яті, а багаторазово перевіряла гіпотези, аналізувала обмеження промπτу («відповідай одним словом») та переконувала себе у правильності відповіді. Користувач бачить лише фінальне слово, проте апаратні ресурси витрачаються на весь обсяг внутрішнього тексту.

2. На яких типах задач міркувальні моделі дають перевагу над інструктивними, а на яких їх використання не виправдане?

Відповідь: Міркувальні моделі забезпечують критичну перевагу в багатокрокових математичних розрахунках, олімпіадній логіці, формальних доказах та дебагу складного коду, де необхідна проміжна самоперевірка (що довів результат 5/5 проти 2/5 у тесті з ручкою). Натомість їхнє використання повністю не виправдане для вилучення простих фактів, класифікації текстів, перекладу або інтерактивних чат-ботів, де приховане міркування лише багаторазово збільшує затримку й фінансові витрати.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Інструктивні моделі навчаються переважно через SFT (Supervised Fine-Tuning) та RLHF на прикладах прямих відповідей «запит — ідеальний результат», навчаючись одразу видавати фінальний текст. Міркувальні моделі проходять масштабне навчання з підкріпленням (RL) на завданнях із верифікованим результатом (математика, код), де алгоритм винагороджується за побудову детальних ланцюжків роздумів, пошук альтернативних шляхів та виправлення власних помилок у процесі генерації.

4. Чи означає більша кількість згенерованих токенів у міркувальної моделі, що вона «розумніша» за інструктивну?

Відповідь: Ні, більша кількість токенів свідчить лише про використання додаткових обчислень на етапі виводу (inference-time compute), а не про більший обсяг фундаментальних знань моделі. На простих питаннях це призводить до так званого «перемудровування» (overthinking), коли система витрачає ресурси даремно, хоча у складних логічних задачах саме наявність проміжних токенів слугує «робочою пам'яттю» для знаходження правильного розв'язку.

5. Як пояснити різницю у швидкості генерації (токенів за секунду) між моделями qwen3:4b та llama3.2:3b?

Відповідь: Швидкість генерації у токенах за секунду насамперед обмежена пропускнуою здатністю оперативної/відеопам'яті (memory bandwidth), оскільки для кожного нового токена необхідно зчитати з пам'яті всі ваги моделі. Компактніша llama3.2:3b має меншу кількість шарів і параметрів (3 млрд проти 4 млрд у Qwen), тому пересилає менше даних на кожному кроці декодування, що забезпечує їй архітектурну перевагу у швидкості інференсу.

Висновок за завданням 1

Експеримент показав, що швидкість генерації та загальний час відповіді є принципово різними речами. Перший показник залежить від заліза та ваги моделі, а другий визначається обсягом тексту, який вона вирішила згенерувати. На питанні про столицю міркування виявилось марнотратством, адже результат однаковий, а витрати токенів зросли майже в 450 разів. На задачі про ручку та зошит ситуація зворотна, оскільки саме покроковий хід думок вберіг від помилки інтуїції та дав 100 відсотків правильних відповідей проти 40 відсотків у швидкої моделі. Тип моделі потрібно підбирати строго під конкретну задачу. Для ланцюжка з 8 послідовних запитів агента різниця складе приблизно 146 секунд проти 23 секунд, через що міркувальна модель абсолютно не підходить для швидкого налагодження.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Таблиця 3 – Вплив system повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	Лаконічний асистент	1375	суржик з англійським и та в'єтнамськими вкрапленнями	12	Ні
llama3.2:3b	українською ,2 речення	276	суржик з англійським и вкрапленнями та граматичними помилками	2	Частково (за кількістю речень вклалася, але мову порушила)
qwen3:4b	Лаконічний асистент	286	Українська	4	Так
qwen3:4b	українською ,2 речення	154	Українська	2	Так

Таблиця 4 – Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	Лаконічний асистент	59	480	539
llama3.2:3b	українською,2 речення	63	91	154
qwen3:4b	Лаконічний асистент	65	3593	3658
qwen3:4b	українською,2 речення	74	992	1066

Відповіді на питання (крок 6)

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Обидві моделі згенерували по два речення, але qwen3:4b впоралася бездоганно, сформулювавши чіткі закінчені думки. llama3.2:3b хоч і вклалася у ліміт, але змішала конструкції та терміни.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Так, llama3.2:3b постійно порушувала мовну інструкцію, вставляючи фрази англійською «on the contrary», «Storage», «temporary» та русизми. Модель qwen3:4b обидва рази надала чистий україномовний текст.

3. Як змінився prompt_tokens при зміні system-повідомлення і чому?

Показник `prompt_tokens` зріс в обох моделях з 59 до 63 у Llama та з 65 до 74 у Qwen. Це сталося через те, що нова інструкція довша за попередню і містить специфічні кириличні токени, які займають більше субодиноць.

4. Чому в `qwen3:4b` значення `completion_tokens` значно більше, ніж у `Llama3.2:3b`, навіть при однаковій довжині видимої відповіді?

`qwen3:4b` є міркувальною моделлю, яка генерує прихований ланцюжок роздумів перед публікацією результату. Поле `completion_tokens` враховує всі токени разом із цим прихованим блоком, тому лічильник фіксує сотні додаткових токенів.

Відповіді на питання (крок 7)

1. Я порівнював інструктивну модель (`Llama3.2:3b`) та міркувальну (`qwen3:4b`) з різними `system`-повідомленнями. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Відповідь: Це зумовлено розміром і якістю тренувального корпусу, а також наявністю етапу попереднього планування. Сімейство Qwen має потужніше представлення багатомовних даних та планує структуру відповіді у внутрішньому ланцюжку думок. LLaMA 3.2 має суто англоцентричну базу, тому при спробі обмежити довжину на нерідній мові її механізм уваги збивається.

2. Чим `system`-повідомлення відрізняється від `user`-повідомлення з точки зору моделі - чи справді воно має більший пріоритет?

Відповідь: Для архітектури трансформера це просто єдиний потік токенів, де ролі розмежовані службовими тегами. Під час інструктивного донавчання модель привчають надавати системним правилам пріоритет над користувацькими. Проте через механізм контекстної уваги останні вхідні дані мають сильний пріоритет «свіжості», тому системний пром프트 не дає абсолютної гарантії поведінки.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Відповідь: Усі додаткові токени витрачаються на прихований внутрішній монолог самоперевірки (блок `thinking`). Апаратний лічильник токенизатора реєструє абсолютно весь згенерований текст від першого до останнього

службового символу. Інструктивна модель одразу генерує фінальні слова, тому її лічильник значно менший.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, - яку вона виконає?

Відповідь: У більшості малих моделей перемагає запит від користувача через ефект близькості останніх токенів. Якщо суперечність є критичною, компактна модель часто втрачає зв'язність: змішує обидві команди, починає повторюватися або переходить на англійську мову.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Відповідь: Моделям малого розміру бракує ємності параметрів для збереження стійких семантичних зв'язків у менш представлених мовах. Одночасне утримання обмежень на довжину, зміст та нерідну мову призводить до збою декодера і підміни кириличних токенів більш імовірними англійськими аналогами.

Висновок за завданням 2

Експеримент довів, що системне повідомлення є лише м'якою настановою, а не надійним програмним обмеженням. Для компактних інструктивних моделей текстової інструкції виявляється недостатньо: модель легко втрачає мовний контроль і продукує суржик. Натомість міркувальна модель завдяки внутрішньому ланцюжку думок значно краще контролює структуру тексту, хоча й генерує колосальний обсяг прихованих токенів. Якщо модель ігнорує системні вимоги щодо формату, покладатися на один лише текст не можна: необхідно застосовувати детерміновані технічні засоби контролю інференсу на кшталт граматик GBNF, схем Pydantic та примусового маскування токенів.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Таблиця 5 – Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	gemini-2.5-flash
Час відповіді, с	3.48	5.13
completion_tokens	480	219
Довжина відповіді, символів	1375	557
Мова відповіді	суржик, змішування з англійською	літературна українська мова
Фактичних помилок у відповіді	твердження, що RAM не записує дані в процесі роботи («не володе можливостями запису»); граматичний брак («додатковм», «пам'яттю»)	0

Відповіді на питання (крок 5)

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Фактично довелося змінити лише три рядки: адресу сервера `base_url`, джерело API-ключа `api_key` через `os.getenv` та назву цільової моделі `model`. Решта структури запиту, обробка параметрів і читання відповідей залишилися абсолютно незмінними завдяки уніфікації інтерфейсу OpenAI.

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Відповідь хмарної моделі значно точніша, лаконічніша та логічно завершена. Наприклад, Gemini чітко розмежувала призначення пристроїв за критерієм енергозалежності волатильності й підсумувала все вдалою аналогією «робочий стіл та шафа», тоді як локальна LLaMA продукувала надлишкові повтори та плутала базові терміни.

3. Чи були в локальній моделі фактичні помилки або проблеми з українською мовою?

Так, локальна модель припустилася грубої фактичної помилки, написавши, що «оперативна пам'ять не володіє можливостями запису даних», що суперечить принципам роботи RAM. Крім того, спостерігався важкий мовний брак: постійне використання суржику, русизмів «пам'ятю», «хранильцем» та вкраплення англійських слів «temporary storage», «on the contrary».

4. Хмарна модель відповіла швидше за локальну - за рахунок чого, якщо запит летить через інтернет?

Хмарні платформи розгортаються на промислових прискорювачах з гігантською пропускнуою здатністю пам'яті, що дозволяє генерувати десятки токенів за секунду навіть для масивних мереж. Висока швидкість обчислень серверного заліза повністю компенсує мережеву затримку ring на передачу пакетів.

.

Відповіді на питання (крок 6)

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Відповідь: OpenAI першою вивела на ринок зручний та масовий REST API для мовних моделей, закріпивши його як фактичний галузевий стандарт. Підтримка цієї структури іншими провайдерами та локальними рушіями дозволяє розробникам перемикатися між хмарою й локальним сервером зміною двох рядків без переписування коду всієї програми.

2. Я порівнював локальну модель на 3 млрд параметрів і хмарну на 70 млрд (або Gemini Flash). Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Відповідь: Кількість параметрів критично визначає ємність «пам'яті» моделі, її словниковий запас і здатність тримати контекст складних граматичних конструкцій нерідних мов. Хоча оптимізація промптів і якість донавчання відіграють вагомую роль, архітектура малого масштабу фізично не здатна уникнути мовних збоїв на складних концептах порівняно з повнорозмірними хмарними системами.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Відповідь: Генерація тексту повністю впирається у швидкість зчитування вагів моделі з пам'яті (memory bandwidth). Корпоративні кластери використовують спеціалізовану пам'ять HBM із пропускнуою здатністю понад 3 ТБ/с та кешування префіксів, що у десятки разів швидше за звичайну оперативну пам'ять DDR4 чи DDR5 у персональному комп'ютері.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Відповідь: Головний ризик полягає у витоку чутливих даних (комерційної таємниці, коду, персональних даних клієнтів) на сторонні сервери або їх використанні для донавчання майбутніх версій моделей. У бізнесі для цього укладають корпоративні угоди з нульовим збереженням даних (Enterprise Zero Data Retention), застосовують маскування чутливих сутностей (PII redaction) або повністю розгортають локальні контури без виходу в мережу.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Відповідь: Це обмеження провайдера на кількість запитів за хвилину (RPM), токенів за хвилину (TPM) або запитів за добу (RPD) для запобігання перевантаженню інфраструктури. Розробники вирішують цю проблему через експоненційну затримку повторів (exponential backoff), черги завдань, асинхронні

пули або розподіл навантаження між кількома різними провайдерами (fallback routing).

Висновок за завданням 3

Локальну модель я оберу у випадках, коли критичними вимогами є конфіденційність даних наприклад, внутрішня документація, медичні картки чи вихідний код закритих проєктів, повна незалежність від стабільності інтернет-з'єднання та відсутність абонплати за трафік.

Хмарну модель доцільно використовувати тоді, коли на першому місці стоїть бездоганна якість мови, суворе слідування структурі відповідей без галюцинацій та робота з багатокроковою бізнес-логікою, де будь-яка граматична чи фактична помилка локальної системи є неприпустимою.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Таблиця 6 – Порівняння локальної та хмарної моделі

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	2.094	2.917	61.7
локальна	llama3.2:3b	середній	2.107	3.225	117.6
локальна	llama3.2:3b	довгий	2.148	4.696	144.8
хмарна	gemini-2.5-flash	короткий	0.970	0.971	33.0
хмарна	gemini-2.5-flash	середній	5.163	5.771	89.6
хмарна	gemini-2.5-flash	довгий	7.032	14.233	288.5

Таблиця 7 – Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	9.094	2.403
Всього, с	9.375	3.231



Відповіді на питання (крок 6)

1. У якої моделі кращий TTFT і чому саме в неї?

На короткому запиті кращий TTFT показала хмарна Gemini, де перший токен з'явився менш ніж за секунду проти двох секунд у локальній моделі, бо вона вже була прогріта на потужних серверах. На довших текстах локальна виявилася швидшою на старті, адже їй не треба витратити час на передачу великого запиту через інтернет.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть?

Ні, не збігається. На короткому промпті Gemini швидше за всіх видала перший токен, але друкувала текст зі скромною швидкістю 33 символи за секунду. На

довгому запиті вона довше думала перед початком, але потім сипала текст із рекордною швидкістю 288 символів за секунду.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промπτу до довгого?

На короткому завданні локальна модель друкувала текст швидше за хмару. На довгому запиті хмара різко вирвалася вперед і стала швидшою майже вдвічі, адже потужне серверне залізо набагато легше справляється з великими обсягами тексту.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

У локальної моделі результати відрізнялися лише на соті частки секунди і були дуже стабільними. У хмарі був відчутний розкид через мережу і навіть спрацьовувало тимчасове блокування через ліміт запитів, тому одного прогону точно замало.

5. Чому холодний старт повільніший за теплий?

Під час холодного старту Ollama змушена читати важкі файли моделі з диска в оперативну пам'ять, що зайняло цілих 9 секунд. При теплому старті модель уже висить у пам'яті комп'ютера, тому видача пішла всього за 2.4 секунди.

Відповіді на питання (крок 7)

1. Що фізично відбувається під час TTFT і чому цей показник вимірюють окремо від загальної швидкості генерації?

Під час TTFT модель обробляє весь отриманий промπτ цілком, перетворює слова на токени та рахує для них таблиці уваги. Окремо його міряють тому, що саме від нього залежить відчуття живої реакції інтерфейсу, адже до появи першої літери користувач просто дивиться у порожній екран.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Хмарі потрібен додатковий час на пересилку пакетів через інтернет та очікування у черзі запитів API. Але коли генерація вже почалася, серверні

відеокарти з широкою шиною пам'яті видають нові токени набагато швидше за звичайну домашню пам'ять.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Холодний старт це стан, коли програма щойно запущена і ваги нейромережі ще лежать на диску, а не в оперативці. На читання файлів моделі витрачається кілька секунд, а під час наступних запусків модель уже висить у RAM і стартує без затримок.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

В один конкретний запуск комп'ютер може відволіктися на фоновий процес, або в мережі підскочить пінг. Середнє арифметичне через такий стрибок спотвориться, а от медіана просто відкидає випадкові збої і показує чесний результат.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Вхідний промпт уже готовий і відеокарта обробляє всі його слова паралельно за один підхід. Відповідь же генерується строго по одному слову за раз: щоразу доводиться перерахувати всі параметри моделі з пам'яті заново, тому вихід завжди повільніший за вхід.

Висновок за завданням 4

Для інтерактивного бота важливіший TTFT. Людині не потрібна космічна швидкість тексту, бо ми все одно читаємо повільно, головне щоб бот зреагував на запит за секунду і не змушував чекати перед пустим полем.

Для фонової обробки великих текстів TTFT взагалі не грає ролі. Якщо програма вночі перебирає тонну документів чи логів, байдуже, чи зачекає вона на старті п'ять секунд, тут вирішує тільки максимальна швидкість генерації символів за секунду.

Завдання 5. Оцінювання якості моделей за рубрикою

Таблиця 8 – Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	2	2	2
2. Формат	0	2	2
3. Стиснення	0	2	2
4. Код	1	2	2
5. Логіка	0	2	2
6. Українська	0	2	2
Разом /12	3	12	12

Таблиця 9 – Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	ні	ні	ні
2. Формат	так	ні	ні
3. Стиснення	так	так	ні
4. Код	так	ні	так
5. Логіка	так	ні	ні
6. Українська	так	ні	ні

Відповіді на питання (крок 6)

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Найбільше балів набрали Qwen та хмарна модель. Основна різниця виникла в логіці та форматі: локальна Llama повністю провалила базову математику та не змогла видати валідний JSON.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

У Llama результати розходилися майже в кожному завданні, вона постійно генерувала різний текст. Це свідчить про вкрай низьку надійність малих моделей за ненульової температури.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Qwen та хмарна модель впоралися ідеально і видали чистий JSON. Llama інструкцію проігнорувала і згенерувала суцільний текст із десятками зайвих слівників.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

У Llama виникли катастрофічні проблеми. Вона використовувала російські слова, ламала відмінки та вигадувала неіснуючі терміни на кшталт "граїшті" або "апарат для стабілізації колег".

Приклад провалу (крок 7)

1. Повний текст запиту:

"8 команд, кожна з кожною по разу. Скільки матчів? При 4 матчах на день скільки днів? Покажи хід розв'язання."

2. Повний текст відповіді:

Давайте разберем проблему поэтапно:\n\n1. Кожна команда грає по разу, тестируя 8 команд.\n2. Кількість матчів не задана, але задана інформація про граїшті змагання.\n3. Поняття \"граїшті\" не ясно, але я здогадую, що це означає, що усі 8 команд грають один проти одного.\n\nПоняття \"граїшті\" не являє себе на мапі, але я здогадую, що у вас наразі є інформація про граїшті змагання. Для простотності, я Assume, що граїшті змагання складається з матчів між командами.\n\n4. Поскольку каждая команда грає по разу, а граєшті складається з матчів, то каждая команда грає один матч против інших 7 команд.\n\n5. Попри те, що змагання складається з 8 команд, а каждая команда грає по разу, то граїшті складається з матчів, які грають усі команди, без урахування одного з них.\n\n6. Кількість матчів в граїшті змаганні можна розрахувати за допомогою формулы: \n\n
$$(8 - 1) * (8 - 2) * \dots * 1$$
 \n\n
$$= 7 * 6 * 5 * \dots * 1$$
 \n\n
$$= 8 * 7!$$
 \n\n
$$= 40320$$
 \n\n7. Тепер, якщо граєшті складається з матчів змагання при 4 матчах на день, то можна розрахувати, скільки днів буде Need для закінчення граєшті.\n\n8. Тривожно велика кількість матчів в граїшті змаганні. На практиці, граєшті не буде виконано за один день, а буде виконано за декілька днів.\n\n9. Для розрахунка, скільки днів буде Need для закінчення граєшті, можна використати формулу:\n\n
$$\text{Дні} = \text{матчі} / \text{матчів на день}$$
 \n\n
$$= 40320 / 4$$
 \n\n
$$= 10080$$
 \n\n10. Таким чином,

для закінчення граєшті змагання потрібно близько 10080 днів.\n\nВажно зазначити, що насправді граєшті складається з багатьох матчів, і не буде виконано за один день."

3. Яка правильна відповідь мала бути:

Кількість матчів рахується за формулою комбінаторики: $8 * 7 / 2 = 28$ матчів. Якщо грати по 4 матчі на день, турнір триватиме рівно 7 днів.

4. Пояснення, чому так сталося:

Модель не зрозуміла суть турнірної сітки і замість комбінацій почала рахувати факторіал. Вона вигадала власну формулу, отримала 40320 матчів і сліпо поділила їх на 4, зробивши висновок, що турнір на 8 команд триватиме 27 років 10080 днів. Це типова галюцинація математичної логіки у малих нейромереж.

Відповіді на питання (крок 8)

1. Які ще підходи до оцінювання використовують на практиці?

Окрім ручних рубрик використовують автоматизовані тести типу MMLU, сліпі голосування користувачів на аренах та метод LLM-as-a-Judge, коли сильна модель перевіряє слабкі.

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown?

У їхніх навчальних даних будь-який код завжди обгорнутий у бектіки. Цей статистичний зв'язок настільки сильний, що модель виконує його на автопілоті.

3. Чому малі моделі гірше пишуть українською?

В їхньому навчанні українських текстів було менше відсотка. До того ж токенизатори погано розбивають наші слова, через що малій моделі просто не вистачає пам'яті на складну граматику.

4. Чому та сама модель дає різні відповіді при повторному запуску?

Через температуру генерації модель випадково обирає наступне слово з кількох варіантів. Зміна одного слова на початку повністю змінює весь подальший текст.

5. Що таке галюцинація і чому модель радше вигадас факт?

Галюцинація це коли модель впевнено пише неправду. Вона не має бази даних для перевірки фактів і просто намагається підібрати найбільш статистично вірогідні слова, щоб бути корисною і не відмовляти користувачу..

Висновок за завданням 5

Оцінка за рубрикою чітко показала, що розмір моделі критично важливий для складних задач. Мала локальна Llama повністю провалила логіку та українську мову, тоді як Qwen показала результати на рівні платної хмарної моделі. Для робочих проєктів я б обрав саме Qwen або хмару.

Завдання 6. Керування випадковістю: temperature і top_p

Таблиця 10 – Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	2	5
temperature	0.7	4	—
temperature	1.5	4	5
top_p	0.1	1	—
top_p	0.5	5	—
top_p	1.0	3	—

Таблиця 11 – Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

Відповіді на питання (крок 7)

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Для генерації JSON або структурованих даних слід обов'язково встановлювати `temperature = 0.0`. Це переводить модель у режим жадібного декодування вибір найімовірнішого токена, що забезпечує максимальну детермінованість та суттєво знижує ризик генерації синтаксично некоректних символів, які можуть зламати парсинг.

2. Яке — генератору варіантів заголовків?

Для генератора заголовків оптимальним є значення `temperature = 0.7`. Як показав експеримент, саме за такого значення модель згенерувала 4 унікальні адекватні варіанти з 5 Bite, QuickBite, FoodBite, HomeBite. Це надає моделі творчу свободу без ризику втратити логіку.

3. Чим вплив `top_p` відрізняється від впливу `temperature` за характером, а не за силою?

`temperature` плавно змінює весь профіль розподілу ймовірностей притискає або розтягує його, тоді як `top_p` працює як жорсткий фільтр, що динамічно відсікає хвіст малоімовірних токенів за порогом сумарної ймовірності, залишаючи лише найбільш надійні варіанти перед випадковим вибором.

4. Чи означає `temperature=0` повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

В теорії `temperature = 0` має гарантувати абсолютну відтворюваність. У Кроці 5 експеримент із 10 запусків дійсно показав, що всі відповіді однакові. Проте на творчому завданні при `temperature=0.0` було отримано 2 унікальні відповіді з 5

QuickBite та HomeBite. Це доводить, що через апаратний недетермінізм обчислень із плаваючою комою на GPU стовідсоткова відтворюваність не завжди гарантується навіть при нульовій температурі.

5. Наведіть найбезглуздішу відповідь, яку модель видала при $\text{temperature}=1.5$, і поясніть, чому вона взагалі могла з'явитися.

При $\text{temperature}=1.5$ модель несподівано видала варіант «Куш» серед ряду стандартних англomовних назв HomeBite, BiteNow, QuickBite. Висока температура штучно згладжує розподіл ймовірностей, наближаючи його до рівномірного. Через це модель починає обирати менш очікувані, рідкісні токени, іноді хаотично перемикаючись на іншу мову чи контекст. Варто зазначити, що модель Qwen виявилася досить стійкою: навіть при 1.5 вона не згенерувала повного марення, а фактичні питання «1896» взагалі залишилися безпомилковими.

Відповіді на питання (крок 8)

1. Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена? Параметр temperature (T) використовується як дільник для логітів (сирих передбачень моделі) перед їхнім перетворенням на ймовірності за допомогою функції softmax . Якщо $T < 1$, знаменник зменшується, логіти збільшуються, і розподіл стає "гострішим" (ймовірність лідерів стрімко наближається до 1). Якщо $T > 1$, логіти зменшуються, і розподіл "згладжується", підвищуючи шанси вибору випадкових токенів.

2. Чим top_p відрізняється від temperature — обидва впливають на різноманітність, але за яким механізмом? Обидва параметри регулюють випадковість генерації, але працюють на різних етапах. temperature змінює саму форму розподілу ймовірностей для всього словника, математично перерозподіляючи вагу. Натомість top_p (nucleus sampling) сортує токени за спаданням ймовірності та залишає лише той набір лідерів, сумарна ймовірність якого дорівнює або перевищує значення p . Решта токенів примусово отримує ймовірність 0.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне? На практиці рекомендується налаштовувати лише один із цих параметрів, залишаючи інший за замовчуванням (наприклад, $\text{temp}=0.7$ і $\text{top_p}=1.0$). Одночасна зміна обох ускладнює передбачення результату, оскільки ви одночасно і деформуєте форму розподілу, і обрізаєте його хвіст, що може призвести до непередбачуваної поведінки моделі.

4. Чому навіть при $\text{temperature}=0$ мовна модель не гарантує абсолютно однакову відповідь при повторних запусках? Сучасні GPU виконують операції матричного множення через тисячі паралельних потоків. Оскільки арифметика з плаваючою комою не є строго асоціативною, зміна порядку додавання на мікрорівні при різних запусках може давати мікроскопічні похибки у фінальних логітах. Якщо два токени мають майже ідентичні бали, ця апаратна мікропохибка може змінити токен-переможця.

5. Які значення temperature використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту? Для витягу структурованих даних, генерації JSON, коду та фактажу використовують значення 0.0 – 0.2. Це мінімізує галюцинації та жорстко фіксує синтаксис. Для творчого тексту (статті, ідеї, назви, брейнштормінг) оптимальними є значення 0.7 – 0.9, що забезпечує різноманітність і гнучкість відповідей без втрати зв'язності.

Висновок за завданням 6

За підсумками експерименту можна зробити висновок, що налаштування параметрів випадковості temperature та top_p є критично важливим для керування поведінкою моделі. Для задач, де потрібна жорстка точність і фактаж як-от видача форматованих даних або історичних дат, слід обов'язково використовувати $\text{temperature}=0.0$, щоб уникнути галюцинацій хоча навіть тоді мікропохибки GPU іноді видають унікальні варіанти. Натомість для творчих завдань, наприклад, вигадкування назв для доставки їжі, підвищення температури до 0.7 робить відповіді цікавими та різноманітними. Використовувати екстремальні значення (1.5) немає

сенсу - генерація деградує, і замість адекватних назв з'являються випадкові слова на кшталт «Куш».

Загальні висновки

Таблиця 12 – Підсумкове порівняння моделей

Критерій	Локальна мала (3В)	Локальна більша (4В/8В)	Хмарна
Швидкість генерації	Висока, модель дуже швидко видає токени навіть на звичайному процесорі.	Середня, потребує більше часу на обробку кожного слова.	Дуже висока, миттєва генерація на потужних серверах.
TTFT	Короткий, відповідь починає друкуватися майже одразу.	Помірний, потребує кілька секунд на роздуми перед першим словом.	Мінімальний, менше секунди за умови стабільного інтернету.
Якість, балів /12	3/12	12/12	12/12
Стабільність формату (завдання 2)	Низька, ігнорує інструкції та видає зайвий текст замість чистого JSON.	Висока, чітко дотримується синтаксису без зайвих символів.	Висока, абсолютно стабільний формат даних.
Потрібні ресурси	Мінімальні, працює на слабких ПК від 4 ГБ оперативної пам'яті.	Середні, від 8 ГБ оперативної чи відеопам'яті для комфортної роботи.	Лише браузер, API-ключ та інтернет-з'єднання.

Вартість	Безкоштовно	Безкоштовно	Безкоштовно в межах лімітів, далі платно.
Куди йдуть ваші дані	Залишаються суворо на локальному комп'ютері.	Залишаються на локальному комп'ютері.	Передаються на зовнішні сервери компанії-провайдера.
Працює без інтернету	Так	Так	Ні
Коли обирати саме її	Для найпростіших завдань на кшталт сумаризації коротких текстів, коли залізо дуже слабке, а дані є суворо конфіденційними.	Для складних завдань з логіки, коду та формату, коли є потужніший комп'ютер і вимагається абсолютна приватність даних.	Для вирішення найскладніших задач, багатоетапних міркувань та чистої української мови, якщо дані не є секретними.

1) Завдяки використанню єдиного стандарту API для локальних та хмарних моделей, перемикання між ними відбувається без переписування базового коду. Це дозволяє легко створювати надійні системи з резервними провайдерами на випадок збоїв інтернету чи лімітів.

2) Оцінювати загальну швидкість моделей потрібно виключно через поділ на час до першого токена та швидкість самої генерації тексту. Змішування цих двох незалежних показників дає хибне уявлення про реальну продуктивність системи.

3) Спеціалізований режим міркування доцільно використовувати лише для складних логічних або багатокрокових задач. Вмикати його для простих фактичних

відповідей чи базового переказу тексту — це марна трата обчислювальних ресурсів та часу.

4) Словесні інструкції в системному промпті не гарантують суворого дотримання формату, оскільки моделі можуть їх проігнорувати і додати зайвий текст. Справжня стабільність структурованих даних, таких як JSON, забезпечується лише через жорстку програмну валідацію.

5) Оскільки великі мовні моделі є принципово недетермінованими системами через апаратні особливості, одиничний запуск не показує об'єктивної картини. Будь-які тести якості чи продуктивності вимагають багаторазових повторень та усереднення результатів.